

Another Type of Question

We don't usually talk about functional $\mathcal{PS}$ -analogues, but there is an important intermediate class you should know about.

Consider the following problem:

INPUT: A 3CNF ϕ .

Suppose we flip a fair coin for every variable. If it comes down *heads*, we set that variable to *True*, otherwise we set it to *False*.

QUESTION: What is the probability that ϕ is set to *True*?

You can see that this probability is just

$$\frac{\text{The \# of satisfying truth assignments to } \phi}{\text{The \# of truth assignments to } \phi}.$$

The Function #3SAT

If ϕ has k variables, the total number of truth assignments is 2^k . So the question boils down to counting the number of truth assignments that satisfy ϕ . So we really want to answer the question:

#3SAT (Sharp-3SAT)

INPUT: A 3CNF ϕ .

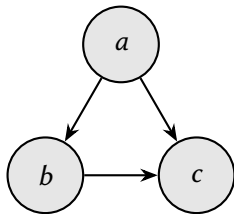
QUESTION: How many satisfying assignments does ϕ have?

This is actually a very important question to us, since it comes up when we try to do statistics on problems that involve, for example, 3CNF formulas. *And a lot of artificial intelligence and machine learning is built around statistical models!*

An Example

As an example, here is a very similar type of question that actually shows up in AI:

A *Bayesian Network* is a graphical model we use to estimate correlated probabilities. It is a DAG, e.g.,



Each node is set to either *true* or *false*, with some probability.

(These nodes can represent, for example, the probability that it is raining today, or that it is wet outside)

An Example (2)

Some nodes are hidden from us — we might not be able to observe the node a , for example. some nodes are observed — we might be able to see the node c , for example. The problem would then be to infer the probability that a is *true*, given our observation of c .

Of course to get this probability we'd need to know how the nodes are correlated — the the graph comes with a set of probability tables, e.g.,

Table for b :

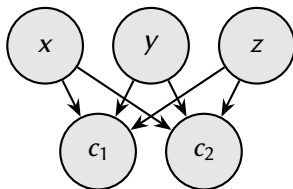
a	<i>true</i>	<i>false</i>
<i>true</i>	0.1	0.9
<i>false</i>	0.7	0.3

Table for c :

a	b	<i>true</i>	<i>false</i>
<i>false</i>	<i>false</i>	0.1	0.9
<i>false</i>	<i>true</i>	0.999	0.001
<i>true</i>	<i>false</i>	0.85	0.15
<i>true</i>	<i>true</i>	1	0

The Relation to 3SAT

We can encode 3SAT into this type of model — we hide the variables and show the clauses, so, for example the formula $(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z)$ would be



We'll set c_1 to *true* with probability 1 if x or z is *true*, or if y is *false*. We'll set c_1 to *false* otherwise. Similarly, we'll set c_2 to *true* with probability 1 iff y or z is *true*, or if x is *false*.

You can see that the above problems are fairly similar — solving one would seem to allow us to solve the other. And of course, once we start using one problem to solve another we're just doing reductions.

The Class $\#\mathcal{P}$

What sort of complexity class would characterize these functions?

In this type of problem:

- We take as an input an instance of a problem in $\mathcal{NP}$.
- We output the *number* of certificates for that instance.

E.G., the number of satisfying truth assignments for a 3CNF formula ϕ .

If $L = \{x \mid \exists c, V(x, c) = \text{True}\}$ is a problem in $\mathcal{NP}$, then $\#L$ is the function that counts the number of these certificates:

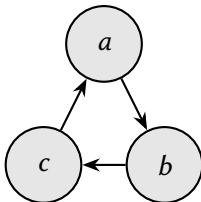
$$\begin{aligned}\#L(x) &= \text{Number of certificates } c \text{ that verify } x \\ &= \left| \{c \mid V(x, c) = \text{True}\} \right|.\end{aligned}$$

The class $\#\mathcal{P}$ (sharp- $\mathcal{P}$) is the class of these functions (and their equivalents):

$$\#\mathcal{P} = \{f : \Sigma^* \rightarrow \mathbb{N} \mid \exists L \in \mathcal{NP}, f = \#L\}.$$

But How to Count Certificates?

Consider the graph



How many Hamiltonian cycles does this have?

Two ways of looking at this question:

- 1 Our verifier accepts (a, b, c) , (b, c, a) , (c, a, b) , so there are three certificates.
- 2 These are all the same cycle, so there's only one certificate.

Either way can work – we'll state in our problem definitions what the certificates are that we're counting.

The Class $\#P$

$\#P$ is:

- Hard for $\mathcal{NP}$.

If we could show that $\mathcal{FP}$ were equal to $\#P$, it would imply that $\mathcal{P}$ was equal to $\mathcal{NP}$.

- We could also tell if a 3CNF ϕ was unsatisfiable, too — so $\#P$ is hard for $\text{co-}\mathcal{NP}$, as well. In fact, it is hard for a much larger set of problems — counting would let us deal with multiple layers of non-determinism. We won't cover the details here.
- $\#P$ is hard for $\mathcal{FNP}$.
- $\mathcal{PS}$ is hard for $\#P$: We can solve problems in $\#P$ by just looping over the possible certificates and counting the number of them that accept.

Question: *Is there such a thing as a $\#P$ -complete problem?*

$\#P$ -completeness

It turns out that there is — a function f is $\#P$ -complete iff:

- $f \in \#P$, and
- Any polytime solution to f would yield a polytime solution for all problems in $\#P$.

But how can we show how to use f to solve any problem in $\#P$?

The trick we'll use here is something called a *parsimonious* reduction...

Remember the last part of question 1 in tutorial 8, where you had to change the reduction to make it one-to-one? That's what we'll be doing here.

A polytime reduction is *parsimonious* if, in addition to being a regular polytime reduction, it preserves the *number* of certificates to the problem.

- Sometimes we can't make a one-to-one relationship, but we can make one that's one-to- k . We'll refer to these as (1-to- k) parsimonious (or k -parsimonious). *This is a slightly non-standard definition.*

#3SAT is $\#\mathcal{P}$ -complete

Recall the proof of Cook's theorem: we spent a class expressing TM operations in terms of a configuration table:

#	q_0	w_0	w_1	...	w_{n-1}	$\sqcup$	$\sqcup$	...	$\sqcup$	#
#	$t_{1,0}$	q_1	w_1	...	w_{n-1}	$\sqcup$	$\sqcup$	...	$\sqcup$	#
#										#
#										#
$\vdots$	$\vdots$	$\vdots$	$\vdots$							$\vdots$
#	$t_{n^k-1,0}$	$t_{n^k-1,1}$	...						t_{n^k-1,n^k-1}	#

#3SAT is $\#P$ -complete (2)

We used this table to build a 3CNF ϕ whose variables were of the form $x_{i,j,s}$, where

$$x_{i,j,s} = \begin{cases} \text{true}, & \text{the cell at } (i, j) \text{ contains the character } s, \\ \text{false}, & \text{otherwise.} \end{cases}$$

Now, if there is only one way to fill the table (i.e., if the TM were deterministic), there would be only one way to set these variables — after all, they just tell us which character is in which cell.

So for a deterministic TM, there is exactly one satisfying truth assignment when the TM accepts, and no satisfying truth assignment otherwise.

If we're dealing with a TM that is a polytime verifier for a language in NP, a similar idea applies, except that there can be lots of certificates.

#3SAT is $\#P$ -complete (3)

We can deal with this by using the NTM N from last week:

Let $N =$ “On input x :

1. Nondeterministically choose certificate c for x .
2. Deterministically run V on $\langle x, c \rangle$.
3. If it accepts, *accept*. Otherwise *reject*.

This will give us a reduction in which there is exactly one satisfying truth assignment of the certificate we choose is accepted, and no satisfying truth assignment otherwise...

This is parsimonious — we get the $\#P$ -completeness of #SAT for free!

#3SAT is #P-complete (4)

To finish off for #3SAT, we just need to fix up the last few steps in the proof where we turned every clause into a series of size-3 clauses:

we rewrote

$$(a \vee b \vee c \vee d)$$

as

$$(a \vee b \vee x) \wedge (\bar{x} \vee c \vee d),$$

Problem: if a , b , c , and d are all true then there are two ways of setting x , but if only one is true there's only one way of setting x .

Solution: Once we're done splitting the clauses, expand them into

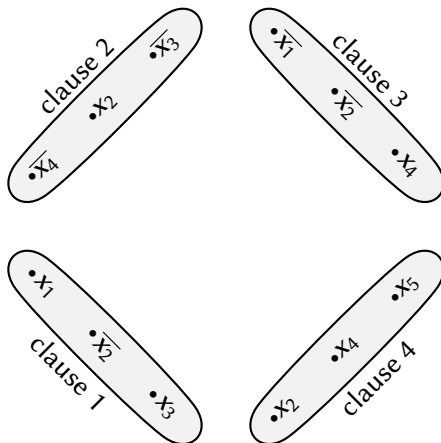
$$(a \vee b \vee x) \wedge (\bar{a} \vee \bar{a} \vee \bar{x}) \wedge (\bar{b} \vee \bar{b} \vee \bar{x}) \wedge (\bar{x} \vee c \vee d).$$

This has the same overall effect, but now there's one possible truth assignment for x .

An Example Parsimonious Reduction: #CLIQUE

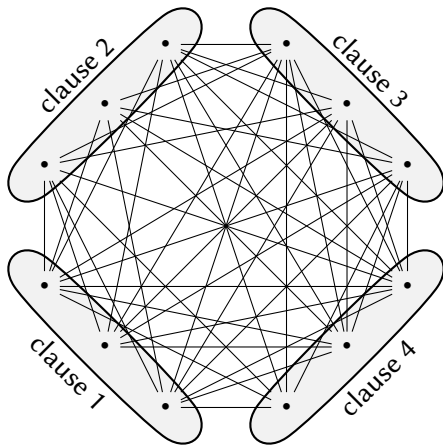
Recall our earlier CLIQUE reduction — e.g., if we start with

$\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_4} \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4) \wedge (x_2 \vee x_4 \vee x_5)$, we'll get the following nodes:



An Example Parsimonious Reduction: #CLIQUE (2)

Now, add edges between different subsets whenever those edges could correspond to some consistent truth formula:



An Example Parsimonious Reduction: #CLIQUE (3)

Now let's look at some of the truth assignments that we can feed into ϕ . Clearly, a non-satisfying assignment like $x_1 = x_2 = x_3 = x_4 = x_5 = F$ can't be matched up with any 4-clique in this graph. That's why the reduction works.

But how many 4-cliques are consistent with the truth assignment $x_1 = x_2 = x_3 = x_4 = x_5 = T$?

Any such clique will need exactly one node from each of the clause groups, and there are:

- Two ways of choosing a true literal from clause 1 (x_1 or x_3).
- One way of choosing a true node from clause 2 (x_2).
- One way of choosing a true node from clause 3 (x_4).
- Three ways of choosing a true node from clause 4 (x_2 , x_4 , and x_5).

So there are six cliques consistent with this truth assignment.

An Example Parsimonious Reduction: #CLIQUE (4)

On the other hand, if we look at the satisfying truth assignment $x_1 = x_2 = x_3 = x_4 = F, x_5 = T$, then there is:

- One way of choosing a true literal from clause 1 ($\overline{x_2}$).
- Two ways of choosing a true node from clause 2 ($\overline{x_1}$ or $\overline{x_3}$).
- Two ways of choosing a true node from clause 3 ($\overline{x_1}$ or $\overline{x_2}$).
- One way of choosing a true node from clause 4 (x_5).

So there are four cliques consistent with this truth assignment.

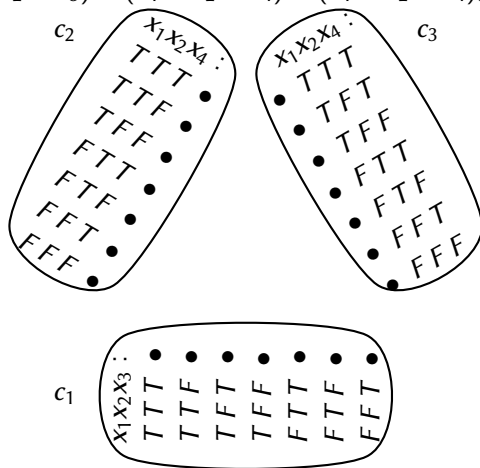
What this means is that if I were to tell you that I've got some other ϕ' , that I've run it through this reduction, and that the resulting G' has 36 k' -cliques, can you tell me how many satisfying truth assignments the 3CNF ϕ' had?

It's actually worse than that — some cliques may be consistent with multiple truth assignments, too! So we really can't use one count to predict the other.

An Example Parsimonious Reduction: #CLIQUE (5)

On the other hand, suppose we used seven nodes per clause: one for each way a truth assignment could satisfy that clause?

E.g., if $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_4)$, we'd get the following nodes:



An Example Parsimonious Reduction: #CLIQUE (6)

- We can see that any satisfying assignment to our input ϕ will be associated with exactly one node per clause group. Since all of these nodes will be consistent, they will form a k -clique. We can also see that non-satisfying truth assignments will not give us any k -clique.
- On the other hand, every k -clique must take one node from every clause group. Since these nodes must be consistent with each other, the variable assignment can be extended to form a satisfying truth assignment for ϕ . Since a variable only shows up in G if it shows up in ϕ , there is exactly one such assignment.
- So there is a one-to-one relationship between the satisfying truth assignments to ϕ and the k -cliques of G . This sort of relationship between certificates is what we're looking for in a *parsimonious* reduction.

A Surprise

It's not too surprising to find that the counting analogue of an $\mathcal{NP}$ -complete language is $\#\mathcal{P}$ -complete.

But there are problems in $\mathcal{P}$ whose counting versions are $\#\mathcal{P}$ -complete, too!

An example would be the language CYCLE:

INPUT: A directed graph $G = (V, E)$.

QUESTION: Is there a cycle in G ?

Just use a DFS algorithm on every vertex, as explained in CSCB63.

But we can reduce to the counting variant CYCLE of this problem from HAM-CYCLE, and so show that it is NP-hard.

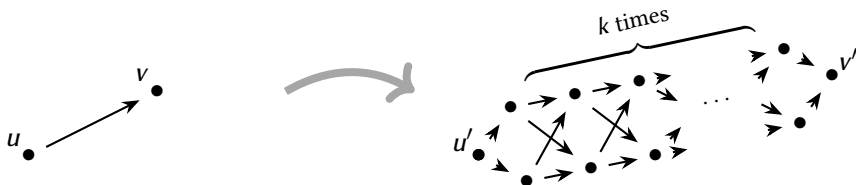
In fact, since you can show that $\#\text{HAM-CYCLE}$ is $\#\mathcal{P}$ -complete, we know that $\#\text{CYCLE}$ is $\#\mathcal{P}$ -complete, as well. The reduction isn't parsimonious, though.

#CYCLE is # $\mathcal{P}$ -complete

Suppose you give me a directed graph G , and want to know if it has a Hamiltonian cycle.

I'm going to take G , and build from it a new graph G' as an input for #CYCLE.

I'll initially set G' to have the same vertices as G . Now, for every edge uv in G , I'll build the following widget:



#CYCLE is # $\mathcal{P}$ -complete (2)

We have k pairs of extra nodes per edge. You can see that, where we originally had one path through this edge from u to v , we now have 2^k — we have to choose which node to pass through in each pair, and this choice is independent.

If we repeat this same construction for every edge in the graph, and keep k constant as we do so, then the exponential blowup in the single edge path that we see above extends to longer paths and cycles: if we want to trace a path of length ℓ in G , we have $2^{\ell k}$ choices about how to do so in G' .

On the other hand, you can see that every cycle in G' corresponds to a cycle in G — take any cycle in G' and “flatten” it back into the edges from G , and you’ll get a cycle.

#CYCLE is #P-complete (3)

Suppose that we set n to be $|V|$. Then, if you have a cycle of length n in G , you'll get at least 2^{kn} cycles in G' .

What if you don't have a cycle of length n ?

Well, in that case the longest cycle you can have is of length $n - 1$. So how many cycle of this form can we find?

If we take any cycle and start listing its vertices (start anywhere you want), you have up to n possibilities for the first, $n - 1$ possibilities for the second, and so on, until we see a repeat, at which point we're done the cycle.

#CYCLE is #P-complete (4)

If we take an upper bound for this number, we get

$$\underbrace{n \times n \times n \times \cdots \times n}_{n-1 \text{ times}} = n^{n-1}.$$

We could also use the factorial, too, but this makes the calculations easier.

So by choosing a good value for k , you can set things up so that a single cycle of length n in G will add more cycles to G' than all n^{n-1} possible cycles of length at most $n - 1$.

A choice of $k = \lceil n^2 \log_2(n) \rceil$ will do the trick.

Finding the Permanent of a Matrix

Another historically important $\#\mathcal{P}$ -complete problem is that of finding the *permanent* of a matrix.

Remember the idea of the *determinant* of a matrix from linear algebra — e.g.,

$$\begin{aligned}\det\left(\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\right) &= \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} \\ &= 1 \times 4 - 2 \times 3 \\ &= -2\end{aligned}$$

and

$$\begin{aligned}\begin{vmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \\ 2 & 3 & 1 \end{vmatrix} &= 1 \begin{vmatrix} 1 & 2 \\ 3 & 1 \end{vmatrix} - 2 \begin{vmatrix} 3 & 2 \\ 2 & 1 \end{vmatrix} + 3 \begin{vmatrix} 3 & 1 \\ 2 & 3 \end{vmatrix} \\ &= 1 \times (1 \times 1 - 2 \times 3) - 2(3 \times 1 - 2 \times 2) + 3 \times (3 \times 3 - 1 \times 2) \\ &= 18\end{aligned}$$

Finding the Permanent of a Matrix (2)

The permanent is the same thing, but we don't flip the signs of the terms for odd permutations:

$$\begin{aligned} \text{perm} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \\ &= 1 \times 4 + 2 \times 3 \\ &= 10 \end{aligned}$$

and

$$\begin{aligned} \text{perm} \begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \\ 2 & 3 & 1 \end{pmatrix} \\ &= 1 \times \text{perm} \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} + 2 \times \text{perm} \begin{pmatrix} 3 & 2 \\ 2 & 1 \end{pmatrix} + 3 \times \text{perm} \begin{pmatrix} 3 & 1 \\ 2 & 3 \end{pmatrix} \\ &= 1 \times (1 \times 1 + 2 \times 3) + 2(3 \times 1 + 2 \times 2) + 3 \times (3 \times 3 + 1 \times 2) \\ &= 54 \end{aligned}$$

Finding the Permanent of a Matrix (3)

The determinant has a nice geometric interpretation in terms of the area of a particular n -dimensional object, and it is useful for problems in linear algebra. The permanent has neither property. But it is useful for counting problems in combinatorics.

Now, we can calculate the determinant for a matrix in a reasonable amount of time — you learned how to do this in linear algebra. But but it turns out that calculating the permanent is $\#\mathcal{P}$ -complete: you can do a reduction from $\#3\text{SAT}$ into the permanent problem for $(0, 1)$ -matrices. We won't go over that reduction, but you should know it exists.

A Small Extra Idea:

Actually, there are several “normal” problems in mathematics whose exact solutions are $\#\mathcal{P}$ -hard to solve...

Suppose we are given a set $S = \{a_1, a_2, \dots, a_n\}$ of non-negative integers.

If we look at the integral

$$\int_{x=0}^{2\pi} \frac{1}{2\pi} \left[\prod_{i=1}^n 2 \cos(a_i x) \right] dx,$$

then by using the identity $\cos \theta = \frac{e^{i\theta} + e^{-i\theta}}{2}$, we can find that our integral counts the number of solutions to the PARTITION instance $\langle S \rangle$!

We don't assume knowledge about complex analysis in this course, so if you don't know this identity don't worry about it. But if you are familiar with it, you may want to go through the calculation to see why this happens.

How Hard are $\#\mathcal{P}$ -complete Problems to Solve?

It's a bit hard to compare a function class like $\#\mathcal{P}$ to language classes like $\mathcal{NP}$, but it seems to be much more powerful.

There is an important theorem called *Toda's theorem* (which we won't be covering in this course) that states that access to a $\#3\text{SAT}$ solver would be enough for us to efficiently solve problems we believe to be far beyond $\mathcal{NP}$.

We can also find a language analogue for $\#\mathcal{P}$, just like we found function analogues for $\mathcal{P}$ and $\mathcal{NP}$:

A language L is in the class $\mathcal{PP}$ (Probabilistic Polynomial Time) iff L has a polytime certificate/verifier pair such that

$$L = \{ \langle x \rangle \mid \text{more than half of the certificates for } x \text{ are accepted.} \}$$

Overall, we know that

$$\mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{PP} \subseteq \mathcal{PS}.$$

We suspect that all of these inequalities are strict.

What About Quantum Computers!?

We've said a lot about problems that we don't think that we can solve in polytime, and one of those problems has been the language

$$\text{FACT} = \{ \langle n, a \rangle \mid n \text{ has a factor } k, \text{ where } 1 < k \leq a \}.$$

And while we don't believe that classical computers can solve this problem efficiently, quantum computers can.

So we can reasonably ask how much quantum computers mess up the complexity section of this class.

It turns out that they don't change things as much as you might hope.

You still have to do $\mathcal{NP}$ -completeness reductions.

To see how they fit into our subject, we remember that quantum computers are, at heart, a type of probabilistic computer. So let's briefly look at a couple of probabilistic classes.

Probabilistic Complexity Classes

Here's the first of our probabilistic complexity classes:

A language L is in the complexity class $\mathcal{PP}$ (Probabilistic Polynomial) iff

$$L = \{ \langle x \rangle \mid \text{more than half of the certificates for } x \text{ are accepted.} \}$$

...But wait, didn't we just cover this class?

We did, but you can see how to reimagine it as a probabilistic class.

If I give you a 3CNF ϕ , then the question: “Does ϕ have a satisfying assignment?” is $\mathcal{NP}$ -complete.

The question: “How many satisfying assignments does ϕ have?” is $\#\mathcal{P}$ -complete.

The question: “If we pick a truth assignment uniformly at random, is the probability that it will satisfy ϕ more than $1/2$?” is $\mathcal{PP}$ -complete.

Probabilistic Complexity Classes (2)

A probabilistic TM is a TM that can flip fair coin as part of its operation.

We can also give the random coins as a type of certificate.

If $L \in \mathcal{PP}$, then it has a polytime probabilistic TM V such that:

- $x \in L \Rightarrow \Pr(V \text{ accepts } x) > 1/2$.
- $x \notin L \Rightarrow \Pr(V \text{ accepts } x) < 1/2$.

Problem: Suppose the probability that V accepts a *yes*-instance x of size n is $1/2 + 1/2^n$ and the probability that V accepts a *no*-instance x of size n is $1/2 - 1/2^n$.

We'd need an exponential number of runs of V
to distinguish between *yes* and *no*-instances!

Probabilistic Complexity Classes (3)

Here's a probabilistic complexity class that isn't as powerful, but is more useable:

The class $\mathcal{BPP}$ (Bounded Probabilistic Polynomial) is defined as the set of languages L that have a polytime probabilistic TM V such that:

- $x \in L \Rightarrow \Pr(V \text{ accepts } x) > 2/3.$
- $x \notin L \Rightarrow \Pr(V \text{ accepts } x) < 1/3.$

If we change the constants $2/3$ and $1/3$ to $1/2 \pm c$ for any constant c , we'll get the same class.

This is the class that we believe best represents the decision problems we can feasibly decide on probabilistic computers.

- We actually suspect that $\mathcal{P} = \mathcal{BPP}$.
- We don't cover this class much in this course since reductions don't interact nicely with $\mathcal{BPP}$.

But What About Quantum Computers?

Well, quantum use quantum mechanics, and quantum mechanics is probabilistic.

If we replace the probabilistic choice of $\mathcal{BPP}$ with a quantum choice, we get the class $\mathcal{BQP}$ (Bounded Quantum Polynomial). A language L is in $\mathcal{BQP}$ iff there is a polytime quantum computer V such that:

- $x \in L \Rightarrow \Pr(V \text{ accepts } x) > 2/3.$
- $x \notin L \Rightarrow \Pr(V \text{ accepts } x) < 1/3.$

We won't define what precisely a quantum computer is here, though.

This is the class that we believe best represents what quantum computers can do efficiently.

The Class BQP

What we know about BQP :

- It contains both $\mathcal{P}$ and BPP .
- It contains $FACT$.
(And a number of other problems involving, e.g., random sampling.)
- It is contained in $\mathcal{PS}$.

What we don't know:

- The relationship between $\mathcal{NP}$ and BQP .

The two classes may well both contain languages not in the other.

So $\mathcal{NP}$ -complete languages are still hard to solve.

The Class BQP

We don't believe that quantum computers can efficiently condition on different outcomes.

(e.g., on a randomly chosen truth assignment satisfying a given 3CNF.)

The class of languages that we could solve with a quantum computer using this sort of post-selection is called $\text{Post}BQP$.

- We believe that BQP is strictly contained in $\text{Post}BQP$.
- We have proven that $\text{Post}BQP = PP$.

What this means: We don't know if quantum computers can efficiently solve $\mathcal{NP}$ -complete problems. But we have a strong reason to believe that they cannot efficiently count their certificates.